# SolariNet Status Panel

Developer reference · firmware, daemon, protocol, and dashboard integration · 21 October 2026

This guide is for engineers changing the Galactic Unicorn panel or its supporting software. The normative contracts remain `status-panel/protocol.h`, `CONTRACT.md`, `CONTRACT-CP.md`, `CONTRACT-AW.md`, and `CONTRACT-SW.md`; this reference explains how those pieces fit and how to verify a change.

> **Observed architecture:** xenon composes fleet data and panel commands in the dashboard API. lithium runs `solariPanel`, which polls over verified HTTPS and transports snapshots/commands to the RP2350 over USB CDC. The firmware renders locally and reports state. An optional authenticated TLS listener supports the network transport path described by the software contract.

## 1. Component boundaries

**xenon · PHP API + SPA**
Aggregates monitoring and inventory data; owns command queue and reported panel state.

→

**lithium · solariPanel**
Authenticates as the panel service user, polls API, validates state, forwards commands, and uploads STATE/CONFIG.

→

**Galactic Unicorn · RP2350**
Maintains display state, history, alarms, configuration, and local controls; renders at 25 Hz.

| Component | Owns | Source of truth |
|---|---|---|
| Dashboard API | Fleet aggregates, alert episode identity, queued controls, last device-reported state, screen configuration | SolariNet monitoring/SoR databases and panel tables |
| Daemon | HTTPS session, API validation, link lifecycle, serial/TLS framing, command forwarding, state/config upload | API for fleet/commands; panel STATE and CONFIG for applied device state |
| Firmware | Physical controls, screen selection, pixel rendering, local history, link status, alarm indication, persisted screen preferences | Latest accepted snapshot plus locally observed controls/ configuration |

Do not duplicate scoring, command completion, or screen configuration authority in another layer. The server computes score and alarm state; the firmware reports consumed command IDs; persistent screen enable/weight values are confirmed by the panel's CONFIG report.

## 2. Wire protocol and compatibility

`status-panel/protocol.h` is normative and `protocol.c` is compiled by both daemon and firmware. The frame uses magic bytes `A5 53`, protocol version `01`, type, little-endian payload length, payload, and CRC-16/CCITT over version through payload. Maximum payload is 2048 bytes; an incomplete frame expires after 500 ms. Parsers resynchronize on magic and reject bad CRCs without treating partial data as a message.

| Direction | Frames | Purpose |
|---|---|---|
| Host → panel | `SNAPSHOT 0x01`, `PING 0x02`, `HELLOREQ 0x03`, `CONTROL 0x04`, `PROVISION 0x05` | Fleet snapshot, link keepalive, capability/state request, idempotent control, USB-only provisioning chunks |
| Panel → host | `HELLO 0x81`, `EVENT 0x82`, `LOG 0x83`, `STATE 0x84`, `CONFIG 0x85`, `PROVACK 0x86` | Firmware identity, local events, diagnostics, consumed command/device state, persisted preferences, provisioning progress |

> **Compatibility rule:** use the shared codec and fixed offsets; never cast a C struct onto wire bytes. Keep unknown frame types skippable, preserve additive payload handling, and make rollout order explicit in the relevant contract before changing a payload or command meaning.

CONTROL commands are re-served until a STATE frame reports `lastCmdId`. A successful serial write does not complete a command. The firmware consumes IDs in ascending order, including invalid commands it rejects; server-side validation should make rejection unreachable in normal operation. A firmware reboot may replay still-pending idempotent commands during the bounded queue window.

## 3. Firmware change map

| Area | Files | Change guidance |
|---|---|---|
| Boot, event loop, alarm and controls | `firmware/main.c`, `panelCtl.c`, `panelPower.c` | Keep input, alarm, power and command state transitions centralized. Hardware-free state logic belongs in testable modules. |
| Link and network transport | `panelLink.c`, `panelNet.c`, `panelNetFsm.c` | Retain sequence ordering, timeout, reconnect and transport gating behavior. Do not reimplement link age arithmetic in render code. |
| Rendering and histories | `panelScreensA/B/C/D.c`, `panelInlay.c`, `panelHist.c`, `panelFont.c` | Render from the environment and elapsed time. Histories advance on accepted snapshots, not on every 25 Hz frame. |
| Persistent settings and provisioning | `panelScreenCfg.c`, `panelProv.c`, `panelProvStore.c` | Preserve validation, generation, CRC, A/B storage and torn-write behavior; test with fake flash before hardware. |
| Physical driver boundary | `panelHw.cpp` | Keep C++ confined to the Pimoroni driver wrapper; the rest of the firmware is portable C. |

The display is 53×11 pixels. Four themes each contain three screens. The current physical behavior, thresholds and intentional deviations are documented in the operator manual and firmware comments; update both whenever visible meaning changes.

## 4. Daemon and API integration

The daemon implementation is standard C in `status-panel/daemon/`. It uses libcurl, OpenSSL, and the vendored cJSON 1.7.19 source. API TLS verification is enabled with the installed CA bundle; HTTP redirects are disabled; requests have bounded connect and total timeouts. The password is read from a dedicated file whose ownership and mode are checked. Keep secrets out of argv, logs, fixtures, and documentation.

- Configuration example: `daemon/solari-panel.conf.example`. The default serial glob is `/dev/serial/by-id/*`; a literal device path is also accepted.
- Systemd unit: `daemon/solari-panel.service`. It runs as `jason`, uses the `dialout` group, restarts after failure, and applies filesystem/kernel restrictions.
- Snapshot cadence defaults to two seconds; API polling defaults to five seconds. Transient API and serial failures preserve last valid data and use retry/backoff paths.
- Optional listener configuration requires an explicit bind address, TLS server certificate/key, client CA and allowed SAN. Wildcard binds are rejected by the daemon.
- The API service principal is read-only for fleet data and has narrowly scoped state/config reporting. Dashboard operators control command creation; UI role checks are not the authorization boundary.

> **Never claim a hardware rollout from host tests.** Deployment acceptance also requires firmware identity/build checks, physical controls, alarm behavior, disconnect/reconnect, API outage recovery, and the applicable cross-lab review.

## 5. Build and test workflow

### Portable daemon

```
cd status-panel/daemon
make clean
make
make test
```

Build flags include C99, `-Wall -Wextra -Werror`. The listener test creates temporary certificates and exercises valid and invalid client identities, CA, certificate purpose, expiration, denylist, and file permissions.

### Firmware state machines and rendering

```
cd status-panel/firmware/test
make clean
make
```

This uses the host compiler only; it does not build a flashable UF2. It covers link ordering/liveness, commands, configuration, provisioning, help, network state, power, renderer safety, and framebuffer parity fixtures with sanitizers on selected suites.

### Hardware firmware build

```
cmake -S status-panel/firmware -B /tmp/solari-panel-build \
  -DPICO_SDK_PATH="$HOME/pico/pico-sdk" \
  -DPIMORONI_PICO_PATH="$HOME/pico/pimoroni-pico" \
  -DCMAKE_BUILD_TYPE=Release
cmake --build /tmp/solari-panel-build -j4
```

Expected artifact: `solari-panel-fw.uf2`. The target is Pico 2 W/RP2350. A host test pass is not a substitute for the Pico toolchain build or device verification. Flashing is an operator action and needs its own explicit authorization and evidence.

## 6. Change checklist

1. Read the owning contract and inspect the current implementation before editing; contracts outrank this summary.
2. For wire changes, update the normative header/codec and both consumers together. Add fixtures that cover old/unknown/malformed cases and sequence behavior.
3. For UI or render changes, update the design reference, firmware renderer, virtual dashboard view, and parity fixtures as applicable.
4. Run the focused host tests, daemon tests, relevant dashboard/API tests, and firmware build for the touched boundary. Record commands and outcomes.
5. Update user/developer manuals and paired PDFs when behavior or operating procedure changes. Keep dated status claims distinct from verified current state.
6. Obtain independent cross-lab review for security, protocol, migration, or live rollout changes. Exercise rollback before live changes; do not infer deployment success from candidate tests.

## 7. Source references and verification limits

Primary contracts: `status-panel/protocol.h`, `status-panel/CONTRACT.md`, `status-panel/CONTRACT-CP.md`, `status-panel/CONTRACT-AW.md`, `status-panel/CONTRACT-SW.md`. Supporting implementation notes: `status-panel/firmware/README.md` and `status-panel/daemon/README.md`.

Related manuals: Panel User’s Guide for screen behavior and controls, and Standalone Runbook for host setup and recovery.

**Observed:** this reference was written against the named candidate source and contracts in the integration worktree on 2026-10-21. **Inference:** the component boundary summary follows those contracts. **UNVERIFIED:** no device build, flash, or live API behavior was exercised while producing this document; tests must be rerun against the final release candidate.